

MLOps, Regression Testing & OpenTelemetry

Complete Study Guide

Data Science & AI Engineering

1. What is MLOps?

MLOps (Machine Learning Operations) = DevOps for Machine Learning. It solves the gap between a data scientist building a model and that model reliably running in production.

The Core Problem MLOps Solves

Without MLOps: A model that gets 95% accuracy on a laptop drops to 70% in production 3 months later — with no way to debug, roll back, or retrain it automatically.

Problem	MLOps Solution
"It works on my machine"	Containerization (Docker)
No experiment history	Experiment Tracking (MLflow)
Can't reproduce results	Model Registry
Model degrades over time	Monitoring & Retraining
Manual deployments	CI/CD Pipelines

The MLOps Lifecycle

Think of it like a factory assembly line for models:

Stage	Description
☐ Data	Collect, validate, and version training data
☐ Experiment	Train models, track params & metrics (MLflow)
☐ Register	Store best model in registry with version tag
☐ Deploy	Serve model via API (FastAPI, Docker, Kubernetes)
☐ Monitor	Track accuracy, latency, data drift in production
☐ Retrain	Auto-trigger new training when performance drops

2. MLflow — Experiment Tracking

MLflow is the industry-standard tool for tracking ML experiments. Think of it as a lab notebook that automatically records everything about each training run.

The 4 Core MLflow Concepts

Concept	Description
Experiment	A project folder grouping related runs (e.g., 'customer-churn-v2')
Run	One training session — records params, metrics, and artifacts
Artifact	The saved deliverable — model file, plots, confusion matrix, etc.
Model Registry	Version control for models: Staging → Production → Archived

Parameters vs Metrics

Type	What it is
Parameter (Input)	Settings you choose BEFORE training: learning_rate=0.01, n_estimators=100
Metric (Output)	Results you measure AFTER training: accuracy=0.93, f1_score=0.91

MLflow Tracking Code

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Set the experiment (creates it if it doesn't exist)
mlflow.set_experiment("customer-churn-prediction")

with mlflow.start_run():
    # Log parameters (inputs)
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)
    mlflow.log_param("learning_rate", 0.01)

    # Train model
```

```
model = RandomForestClassifier(n_estimators=100, max_depth=5)
model.fit(X_train, y_train)

# Log metrics (outputs)
accuracy = accuracy_score(y_test, model.predict(X_test))
mlflow.log_metric("accuracy", accuracy)
mlflow.log_metric("f1_score", f1)

# Save model as artifact
mlflow.sklearn.log_model(model, "model")

# View all experiments in browser
# Run: mlflow ui
# Open: http://localhost:5000
```

Model Registry — Promoting Models to Production

The Model Registry is like a hiring pipeline: Staging (being tested) → Production (live) → Archived (retired).

```
import mlflow
from mlflow.tracking import MlflowClient

# Register a model from a run
mlflow.register_model("runs:/abc123xyz/model", "ChurnPredictor")

# Promote version 3 to Production
client = MlflowClient()
client.transition_model_version_stage(
    name="ChurnPredictor",
    version=3,
    stage="Production"
)

# Load the production model anywhere in your codebase
model = mlflow.sklearn.load_model("models:/ChurnPredictor/Production")
predictions = model.predict(new_data)
```

3. Regression Testing for ML Models

ML regression testing answers: 'Did my new model version break anything that was working before?' Like a safety checklist before flying a plane — you check every system before going live.

NOTE: This is NOT regression analysis (predicting continuous values). It's regression = going backwards, i.e., checking you didn't break previous performance.

Types of Regression Tests

Test Type	What It Catches
Accuracy Threshold	New model must be $\geq$ baseline accuracy or it fails CI/CD
Latency Test	Prediction must return in $< X$ milliseconds
Data Drift Test	Input data distribution hasn't shifted significantly
Slice Testing	Model performs fairly across subgroups (age, gender, region)
Output Schema Test	Model output format/shape hasn't changed unexpectedly

Regression Test Code

```
import mlflow.sklearn
from sklearn.metrics import accuracy_score
import time
import pytest

# Load model from registry
@pytest.fixture
def model():
    return mlflow.sklearn.load_model("models:/ChurnPredictor/Production")

# Test 1: Accuracy must be above threshold
def test_model_accuracy(model):
    accuracy = accuracy_score(y_test, model.predict(X_test))
    assert accuracy >= 0.85, f"Accuracy {accuracy:.3f} below threshold 0.85"

# Test 2: Prediction speed must be fast enough
def test_prediction_latency(model):
    start = time.time()
    model.predict(X_test[:1]) # Single prediction
    latency = time.time() - start
    assert latency < 0.1, f"Too slow: {latency:.3f}s (threshold: 0.1s)"

# Test 3: No data drift in key features
def test_no_data_drift():
```

```
from scipy import stats
stat, p_value = stats.ks_2samp(train_data['age'], new_data['age'])
assert p_value > 0.05, f"Data drift detected in age feature (p={p_value:.4f})"

# Test 4: Slice testing (fairness)
def test_slice_performance(model):
    for group in ['18-30', '31-50', '51+']:
        mask = X_test['age_group'] == group
        acc = accuracy_score(y_test[mask], model.predict(X_test[mask]))
        assert acc >= 0.80, f"Low accuracy for group {group}: {acc:.3f}"
```

CI/CD Pipeline for ML Models

Regression tests are run automatically in a CI/CD pipeline before promoting a model to production:

Pipeline Stage	What Happens
1. Train	New model trained with updated data or hyperparameters
2. Register	Model saved to MLflow registry as 'Staging'
3. Test	All regression tests run automatically (pytest)
4. Gate	If ALL tests pass → promote to Production; else → fail + alert
5. Deploy	Production model served via FastAPI / Docker container
6. Monitor	OpenTelemetry tracks live performance continuously

4. OpenTelemetry (OTel) — Observability for AI

OpenTelemetry is the industry-standard framework for observability — it's like a black box flight recorder for your AI application. It captures everything that happens in production so you can debug issues after the fact.

The 3 Pillars of Observability

Pillar	Analogy	What It Captures
Traces	Journey map of one request	Full path end-to-end: user request → RAG → LLM → response
Metrics	Dashboard gauges over time	Request counts, error rates, avg latency, token usage
Logs	Diary entries with timestamps	Detailed text records: prompts, responses, errors, debug info

Why OTel Matters for AI/LLM Apps

Thing You Want to Track	Why It Matters
LLM call latency	Detect slow responses before users complain
Token count per request	Monitor and control API costs
RAG retrieval time	Identify slow vector search queries
Agent loop iterations	Detect infinite loops or runaway agents
Error rates by endpoint	Know which API routes are failing
Prompt + response content	Debug hallucinations and wrong answers

OpenTelemetry Core Concepts

Concept	Description
Span	A single unit of work with a start time, end time, and attributes
Trace	A collection of spans that together represent one full request journey

Tracer	The object you use to create spans in your code
Exporter	Where telemetry data is sent: console, Jaeger, Prometheus, Datadog
Processor	Handles batching/filtering before export (BatchSpanProcessor)

OTel Setup & Instrumentation Code

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import (
    ConsoleSpanExporter,
    BatchSpanProcessor
)

# 1. Setup - configure where traces go
provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(ConsoleSpanExporter()) # Send to console
    # In production: use JaegerExporter or OTLPExporter
)
trace.set_tracer_provider(provider)

# 2. Create a tracer for your app
tracer = trace.get_tracer("my-ai-app")

# 3. Instrument your LLM call
with tracer.start_as_current_span("llm-inference") as span:
    # Tag the span with useful metadata
    span.set_attribute("llm.model", "llama3-70b-8192")
    span.set_attribute("llm.prompt_tokens", 150)
    span.set_attribute("llm.temperature", 0.7)

    # Your actual LLM call
    response = llm.invoke("Summarize this document...")

    # Record the output
    span.set_attribute("llm.completion_tokens", 200)
    span.set_attribute("llm.total_tokens", 350)
    span.set_attribute("llm.latency_ms", 340)
```

Tracing a Full RAG Pipeline

```
with tracer.start_as_current_span("rag-pipeline") as root:
    root.set_attribute("user.query", query)

    # Span 1: Embedding
    with tracer.start_as_current_span("embed-query") as embed_span:
```

```
query_vector = embedder.encode(query)
embed_span.set_attribute("embedding.model", "BAAI/bge-small-en-v1.5")

# Span 2: Vector Search
with tracer.start_as_current_span("vector-search") as search_span:
    chunks = vector_store.search(query_vector, top_k=5)
    search_span.set_attribute("retrieval.chunks_found", len(chunks))
    search_span.set_attribute("retrieval.top_k", 5)

# Span 3: LLM Generation
with tracer.start_as_current_span("llm-generate") as llm_span:
    answer = llm.invoke(build_prompt(query, chunks))
    llm_span.set_attribute("llm.tokens_used", count_tokens(answer))
```

Metrics with OpenTelemetry

```
from opentelemetry import metrics
from opentelemetry.sdk.metrics import MeterProvider

# Setup metrics
meter_provider = MeterProvider()
metrics.set_meter_provider(meter_provider)
meter = metrics.get_meter("my-ai-app")

# Create metric instruments
request_counter = meter.create_counter(
    "llm.requests.total",
    description="Total LLM API calls made"
)

latency_histogram = meter.create_histogram(
    "llm.latency.ms",
    description="LLM response latency in milliseconds"
)

token_counter = meter.create_counter(
    "llm.tokens.used",
    description="Total tokens consumed"
)

# Use in your code
request_counter.add(1, {"model": "llama3-70b", "endpoint": "/chat"})
latency_histogram.record(340, {'model': 'llama3-70b'})
token_counter.add(350, {'model': 'llama3-70b', 'type': 'total'})
```

5. The Complete MLOps Loop

Here is how MLflow + Regression Testing + OpenTelemetry work together in a production ML system:

Step	Tools & Actions
1. Train & Track	MLflow logs all params, metrics, artifacts for every run
2. Compare & Register	Pick best run in MLflow UI → register as 'Staging' version
3. Regression Test	pytest suite checks accuracy, latency, drift, fairness
4. Gate Decision	All tests pass → CI/CD promotes model to 'Production'
5. Deploy	Docker container serves model via FastAPI endpoint
6. Monitor with OTel	Traces + metrics collected for every prediction request
7. Alert & Retrain	OTel alert fires when accuracy drops → triggers retraining pipeline

Tool Stack Summary

Tool	Category	Job It Does
MLflow	Experiment Tracking	Records all runs, params, metrics, models
MLflow Registry	Model Versioning	Staging → Production promotion workflow
pytest	Regression Testing	Automated pre-deployment model validation
OpenTelemetry	Observability	Traces, metrics, logs for live production
Docker	Containerization	Consistent environment for model serving
Jaeger / Prometheus	Backends	Storage and visualization for OTel data

6. Q&A Study Cards

Q: What problem does MLOps solve, and why can't a data scientist just hand off their model?

A: MLOps solves the gap between model development and production reliability. Data scientists optimize for accuracy on their local data; production models face changing data distributions, infrastructure differences, no versioning, no monitoring, and no rollback plan. MLOps provides the tooling and processes to bridge this gap reliably.

Q: What is the difference between a parameter and a metric in MLflow?

A: A parameter is an input you set BEFORE training (e.g., `learning_rate=0.01`, `n_estimators=100`). A metric is an output you measure AFTER training (e.g., `accuracy=0.93`, `f1_score=0.91`). Parameters go into `log_param()`; metrics go into `log_metric()`.

Q: What is the MLflow Model Registry and what are its 3 stages?

A: The Model Registry is version control for ML models — like GitHub but for model files. The 3 stages are: Staging (model is being evaluated/tested), Production (model is live and serving users), and Archived (retired model kept for audit/rollback purposes).

Q: What is ML regression testing? How is it different from statistical regression?

A: ML regression testing checks whether a new model version broke performance that was previously working — like a CI/CD safety gate. Statistical regression is a prediction algorithm (predicting continuous values). The word 'regression' here means 'going backwards' — checking you haven't regressed to worse performance.

Q: What are the 4 key types of ML regression tests you should run before deploying a model?

A: 1) Accuracy Threshold — new model must meet or exceed baseline accuracy. 2) Latency Test — prediction must return within acceptable milliseconds. 3) Data Drift Test — incoming data distribution must not have shifted significantly. 4) Slice Testing — model must perform fairly across subgroups (age, gender, region).

Q: What are the 3 pillars of observability in OpenTelemetry?

A: 1) Traces — a journey map of one complete request end-to-end (shows every step and its timing). 2) Metrics — aggregated dashboard gauges over time (request counts, error rates,

avg latency). 3) Logs — detailed timestamped diary entries for debugging (prompts, responses, errors).

Q: What is a Span in OpenTelemetry, and how does it relate to a Trace?

A: A Span is a single unit of work with a start time, end time, and attributes (like 'llm.tokens_used'). A Trace is a collection of related spans that together represent one full request journey — for example: [embed-query span] + [vector-search span] + [llm-generate span] = one RAG pipeline trace.

Q: Why is OpenTelemetry especially important for LLM/AI applications compared to regular web apps?

A: LLM apps have unique cost and quality concerns: token usage directly translates to API costs, model latency affects user experience, RAG retrieval quality affects answer accuracy, and agent loops can run indefinitely. OTel lets you track all of these: token count, latency per component, retrieval chunk counts, loop iterations — giving you full visibility into both cost and quality.

Q: How do MLflow, regression testing, and OpenTelemetry fit together in the MLOps pipeline?

A: MLflow handles the pre-deployment phase (tracking experiments, registering models). Regression tests act as the deployment gate (must pass before going live). OpenTelemetry handles the post-deployment phase (monitoring live performance). Together they form a complete loop: train → track → test → deploy → monitor → retrain.

Q: What triggers a model retraining in a production MLOps system?

A: Common triggers include: 1) Scheduled retraining (weekly/monthly regardless of performance). 2) Performance alerts — OpenTelemetry/monitoring detects accuracy dropping below threshold. 3) Data drift alerts — new incoming data distribution diverges significantly from training data. 4) Manual trigger by ML engineer after investigating an issue.

7. Quick Reference Cheat Sheet

MLflow Key Functions

Function	What It Does
<code>mlflow.set_experiment('name')</code>	Set/create an experiment
<code>mlflow.start_run()</code>	Start recording a new run (use as context manager)
<code>mlflow.log_param('key', value)</code>	Record a training parameter (input)
<code>mlflow.log_metric('key', value)</code>	Record a model metric (output)
<code>mlflow.sklearn.log_model(model, 'name')</code>	Save the model as an artifact
<code>mlflow.register_model(uri, name)</code>	Register a run's model to the registry
<code>client.transition_model_version_stage()</code>	Promote model to Staging/Production/Archived
<code>mlflow.sklearn.load_model('models:/Name/Stage')</code>	Load a registered model
<code>mlflow ui</code>	Launch web dashboard at localhost:5000

OpenTelemetry Quick Setup

```
# Minimal OTEL setup for an AI app
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import ConsoleSpanExporter, BatchSpanProcessor

provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(ConsoleSpanExporter()))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("app-name")

# Instrument any function
with tracer.start_as_current_span("operation-name") as span:
    span.set_attribute("key", "value")
    result = do_work()
```

MLOps Terms Glossary

Term	Meaning
MLOps	DevOps practices applied to ML model lifecycle
MLflow	Open-source experiment tracking and model registry tool
Experiment	Named project folder grouping multiple training runs
Run	Single training session recorded in MLflow
Artifact	Any file produced by a run (model, plot, data sample)
Model Registry	Versioned store for models with Staging/Production/Archived stages
Data Drift	When production input data distribution diverges from training data
Slice Testing	Testing model performance on demographic subgroups
Observability	Ability to understand system state from its outputs (traces/metrics/logs)
Span	A single timed operation in OpenTelemetry
Trace	End-to-end journey of one request, made up of spans
Exporter	Component that sends telemetry data to a backend (Jaeger, Prometheus)